

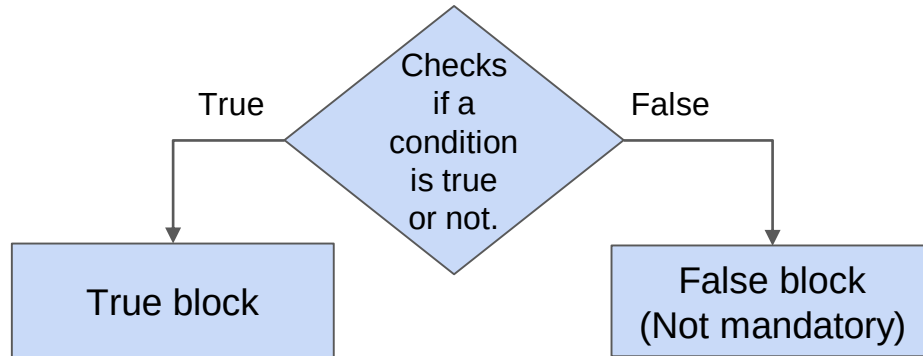
Decision Making

Branching or Selection Statements

Conditional / Selection Statement

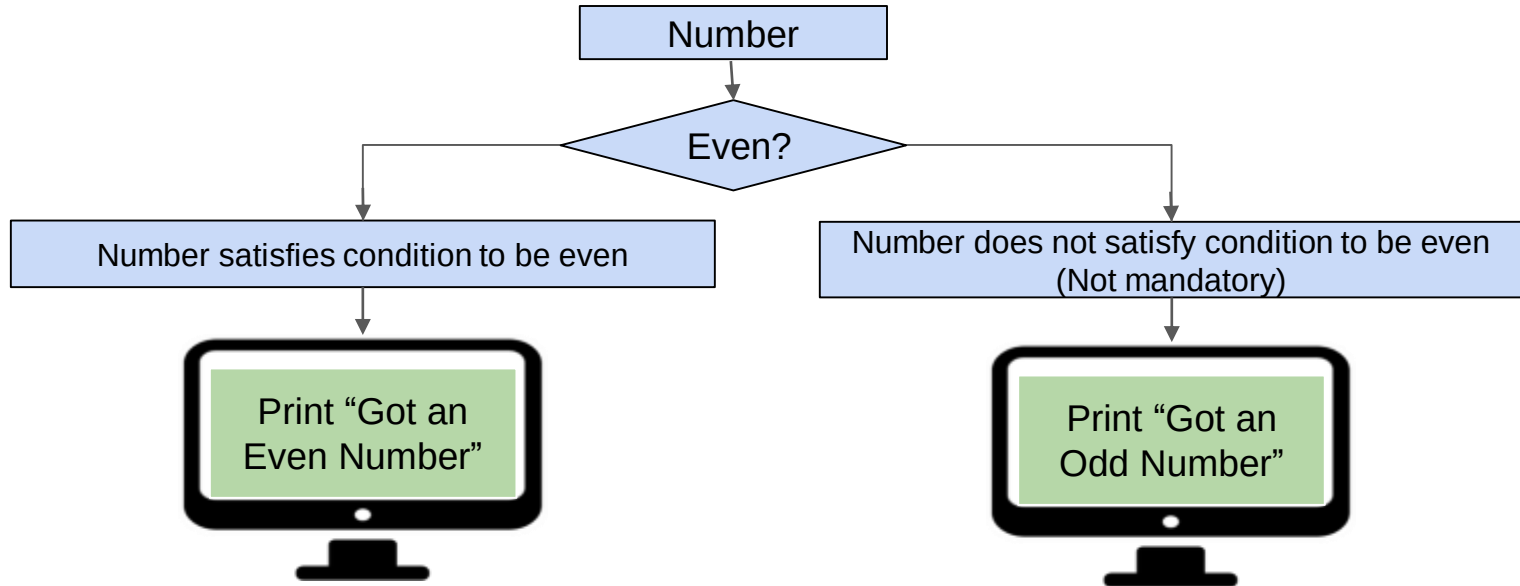
- Execution of a block of code on the basis of some **Conditions**.
- Non sequential program statements
- Selection statements may be **unary or non unary**.
- Ex: if, else if, else, switch-case

Remember this diamond shape from the “Flowchart” chapter?



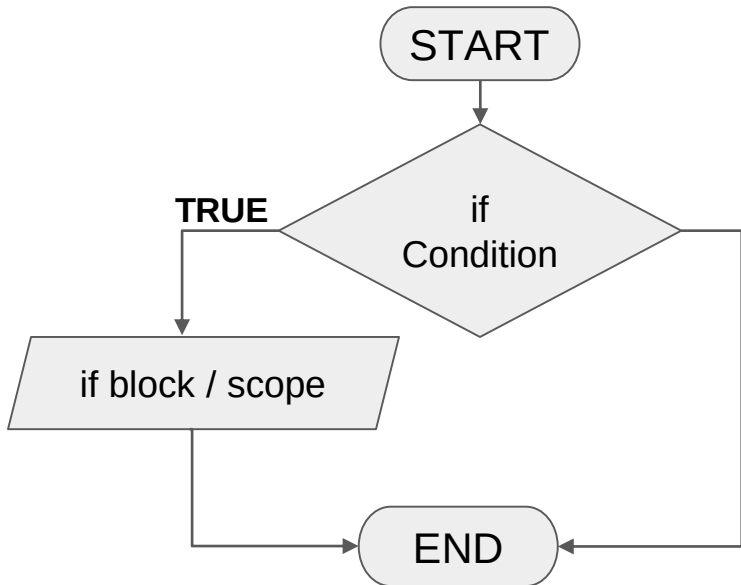
Conditional / Selection Statement (Contd)

- Ex: Printing “Got an Even Number” only if a number satisfies condition to be even or Print “Got an Odd Number” if a number did not satisfy condition to be even.



if Selection Statement

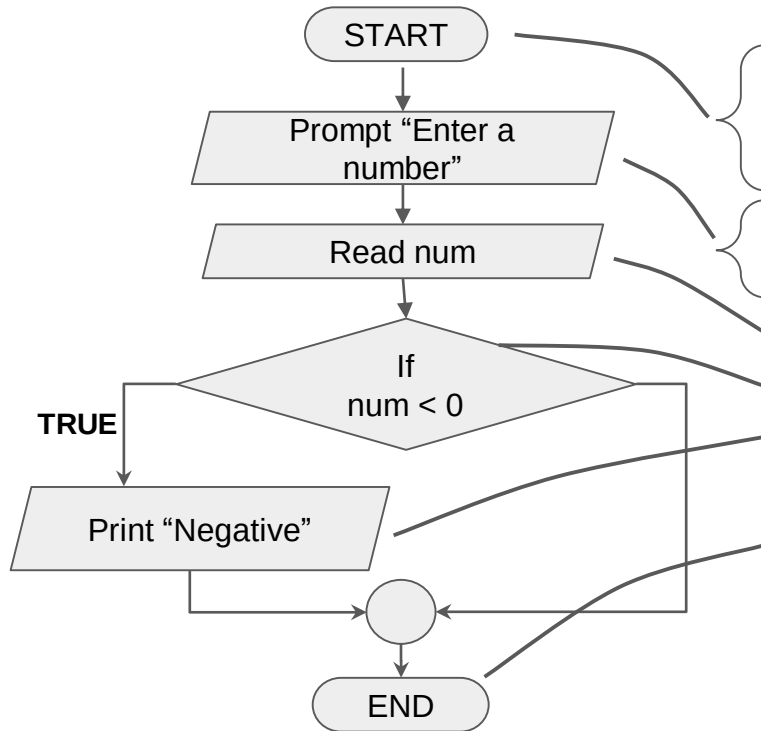
- Unary selection statement
- Will execute block of code only if condition is **True**.



```
1. class Branching{
2.     public static void main (String[] args)
3.     {
4.         if (condition) {
5.             if condition block;
6.         }
7. }
```

if Selection Statement (Contd)

Based on the number input, print whether the number is negative.



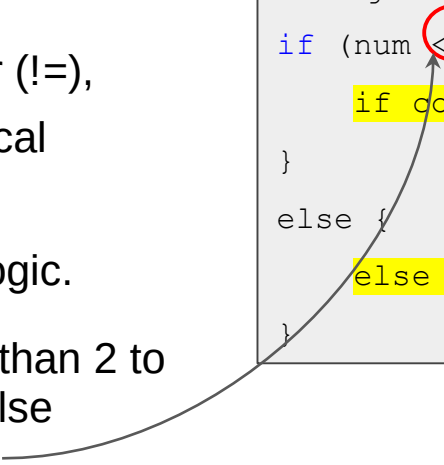
```
1. import java.util.Scanner;
2. class NumberCheck{
3.     public static void main (String[] args) {
4.         Scanner sc = new Scanner(System.in);
5.         System.out.print("Enter a number: ");
6.         int num = sc.nextInt();
7.         if (num < 0) {
8.             System.out.println("Negative");
9.         }
10.        sc.close();
11.    }
12. }
```

Checking Conditions using Operators

- Used to evaluate expressions and return a boolean value (true or false).
- **Conditional operators in Java:**
Equality Operator (==), Inequality Operator (!=), Comparison Operators (<, >, <=, >=), Logical Operators: NOT (!), AND (&&), OR (||).
- Essential for creating effective branching logic.

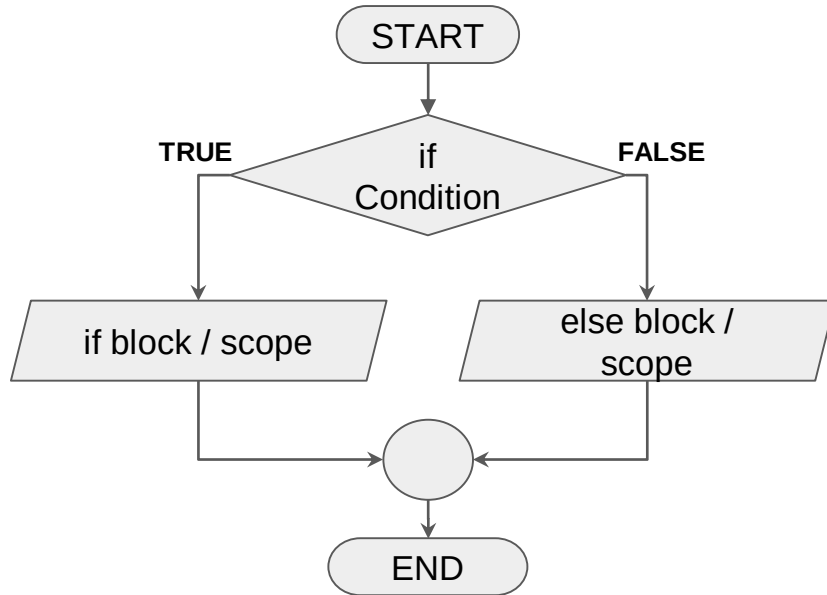
Example: Checking if a number is equal or less than 2 to execute if condition block, otherwise executes else condition block.

```
Scanner sc = new  
Scanner(System.in);  
String num = sc.nextInt();  
if (num <= 2) {  
    if condition block;  
}  
else {  
    else condition block;  
}
```



if - else Double Selection Statements

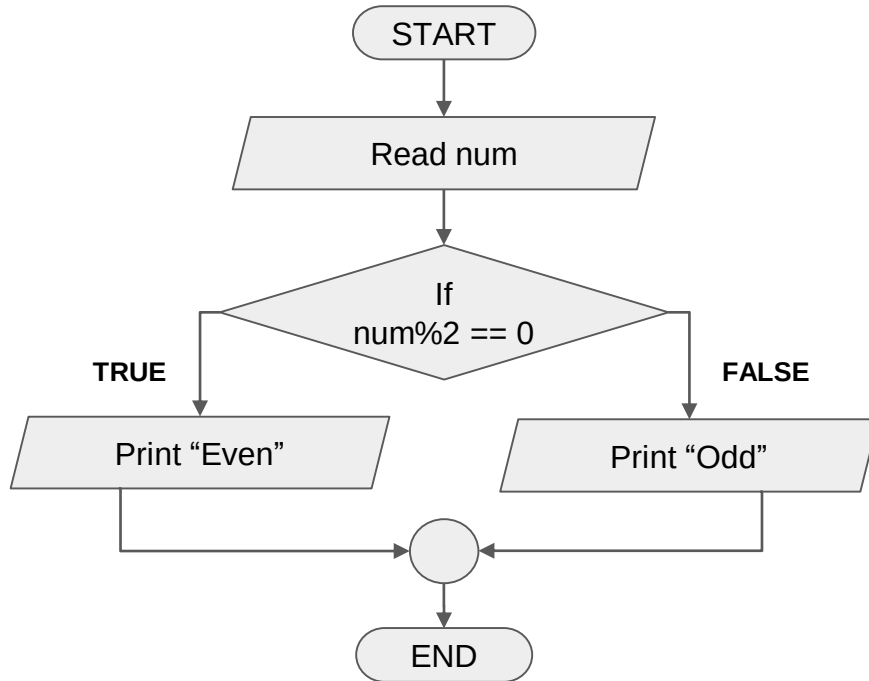
- else block executed when if condition yields **False**.
- else does not have any condition.



```
1. class DoubleSelection {  
2.     public static void main (String[] args)  
3.     {  
4.         if (condition) {  
5.             if condition block;  
6.         }  
7.         else {  
8.             else condition block;  
9.         }  
10. }
```

if - else Double Selection Statements (Contd)

Print “Even” or “Odd” based on user given input.

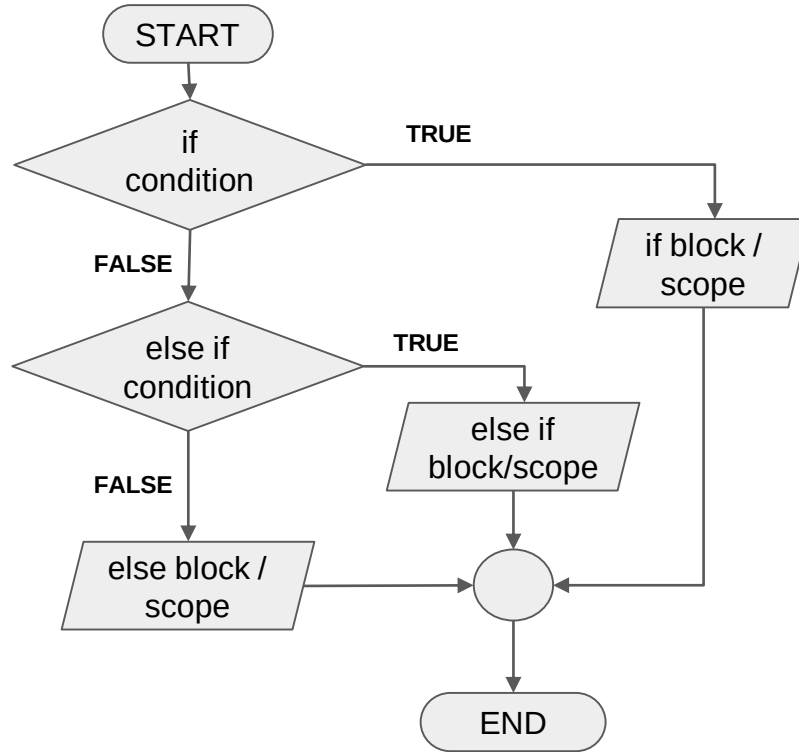


```
1. Scanner sc = new Scanner(System.in);
2. int num = sc.nextInt();
3. if (num%2 == 0) {
4.     System.out.println("Even");
5. }
6. else {
7.     System.out.println("Odd");
8. }
```

if - else if - else Multiple Selection Statements

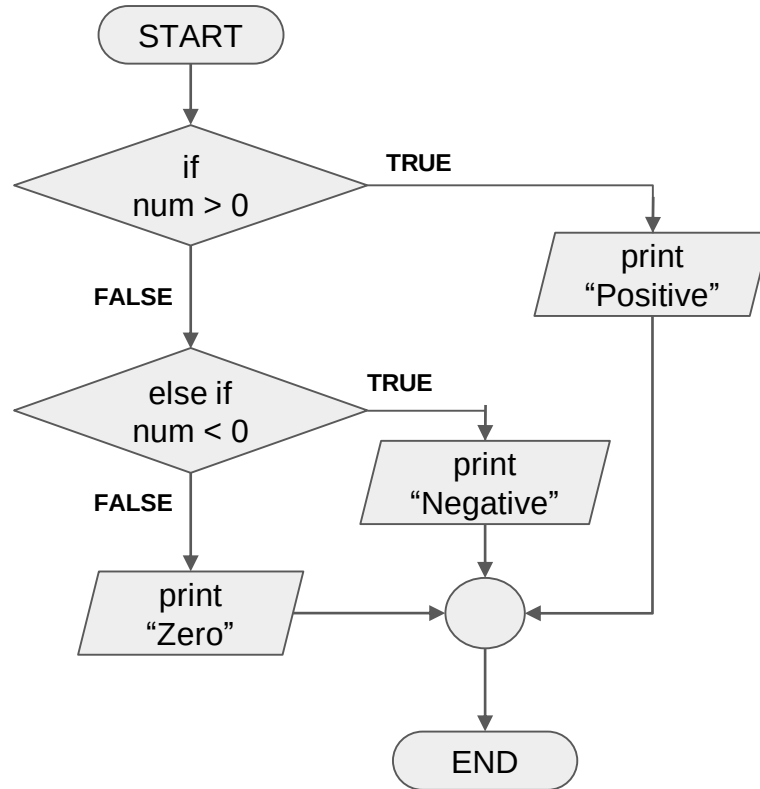
- Follows top down approach.
- There can be multiple else if conditions.
- else if and else cannot exist without if condition.
- Only the first True conditional block executed. Following conditions are not checked.
 - **Use of elif/ else:** Evaluating other conditions after if statement.

if - else if - else Multiple Selection Statements (Contd)



```
1. class MultiBranching {
2.     public static void main (String[] args) {
3.         if (condition) {
4.             if condition block;
5.         }
6.         else if (condition) {
7.             else if condition block;
8.         }
9.         else {
10.            else condition block;
11.        }
12.    }
13. }
```

if - else if - else Multiple Selection Statements (Contd)



Check if a number is positive or negative or equal to 0.

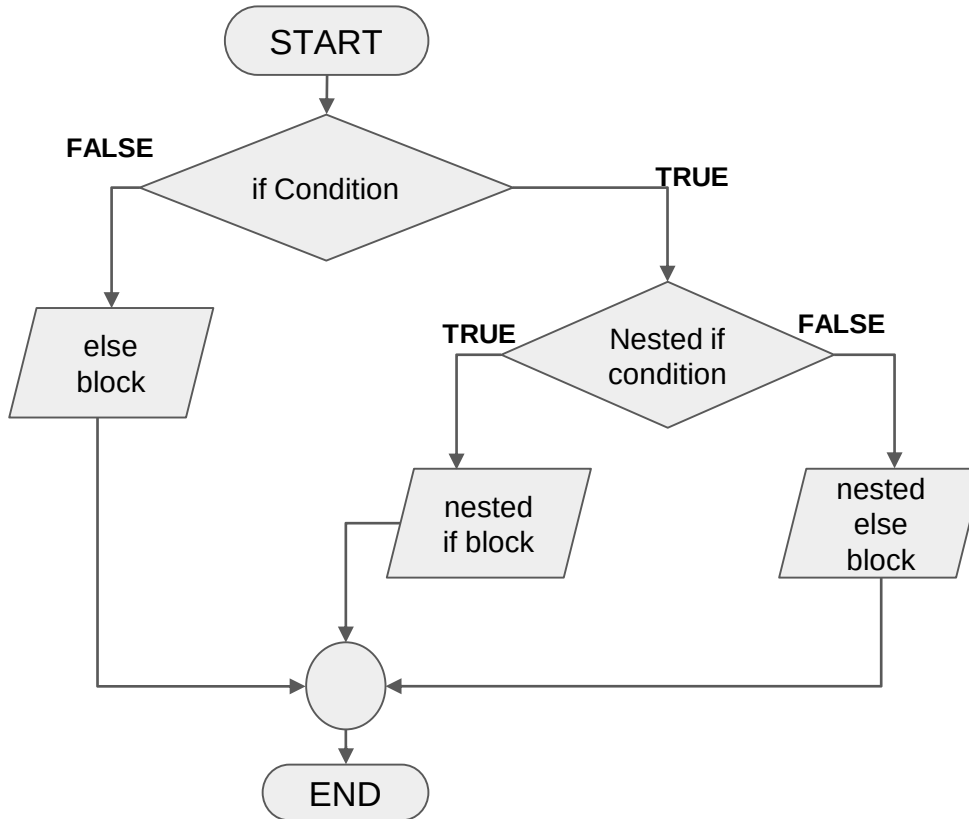
```
1. int num = -5;
2. if (num > 0) {
3.     System.out.println("Positive");
4. }
5. else if (num < 0) {
6.     System.out.println("Negative");
7. }
8. else {
9.     System.out.println("Zero");
10. }
```

Nested Decision Making

- A conditional statement contained inside another conditional statement.
- Statements inside are only evaluated if outer branching statements are true.
 - **Use:** Building complex layered logic.

Use case: Checking if an even number is a prime number will require nested if statements. Outer if statement will check if the number is even or not and the inner if will check if the number is prime or not.

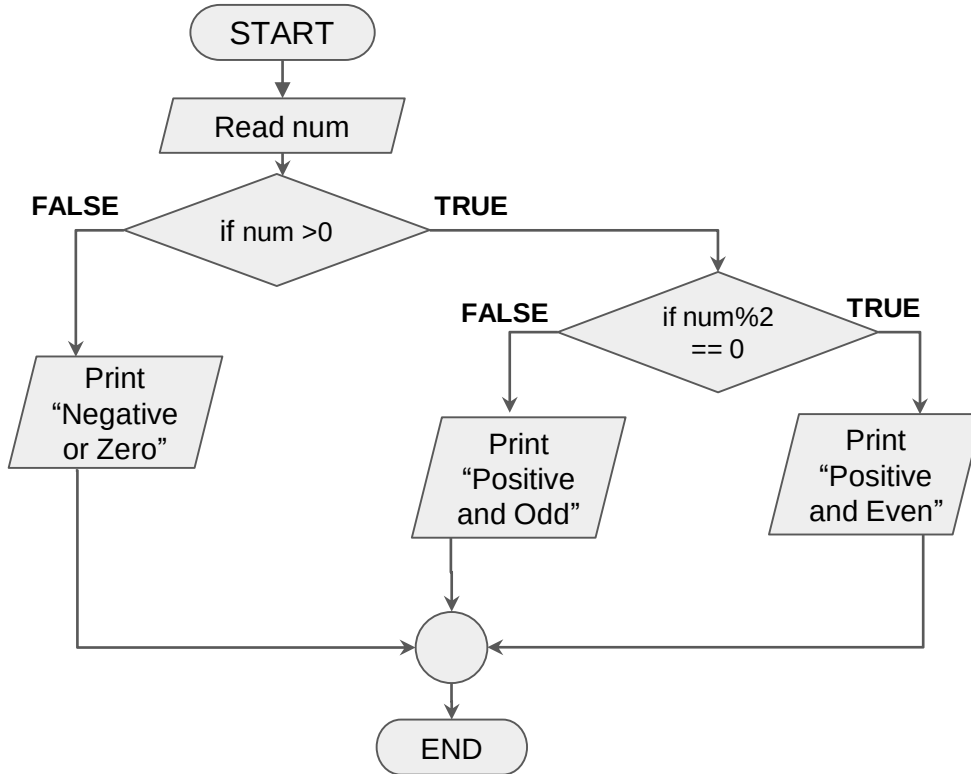
Nested Decision Making (Contd)



```
1.  if (condition) {  
2.      if (condition) {  
3.          nested if inside if block;  
4.      }  
5.      else {  
6.          nested else inside if block;  
7.      }  
8.  }  
9.  else {  
10.     else block;  
11. }
```

Nested Decision Making (Contd)

Design a flowchart that reads a number and prints whether it is positive, negative or zero. If positive, also print whether it is Even or Odd..



```
1. Scanner sc = new Scanner(System.in);
2. int num= sc.nextInt();
3. if (num>0) {
4.     if (num%2==0) {
5.         System.out.println("Positive and Even");
6.     }
7.     else {
8.         System.out.println("Positive and Odd");
9.     }
10. }
11. else{
12.     System.out.println("Negative or Zero");
13. }
```

THANK YOU!